

COMPLETE DOCUMENTATION

Field-Service Job Workflow

Technician Assignment · Start/Complete Work · Materials & Stock · Photos · Customer Sign-off ·
Auto-Invoicing

CRM Module Documentation — Three Audiences

Prepared: 24 August 2026

Status: Live — extends the existing Appointments module

Contents

Part 1 — For Sellers · pitching this to field-service businesses

Part 2 — For End Users · day-to-day usage guide (Tenant Admin / Technician)

Part 3 — For Developers · architecture, data model, roadmap



For Sellers

How to pitch, position, and sell the Field-Service Job Workflow

1.1 The 30-Second Pitch

Any business that sends a person to a customer's site — an electrician, a plumber, an AC repair technician, an installer — needs proof of what happened: who went, when they arrived, what they did, what they used, and that the customer actually approved the work. This CRM now runs that entire chain inside the same Appointments feature the tenant already uses to book jobs — no separate app, no separate login.

ONE-LINER FOR THE PROSPECT

"Book the job, assign a technician, they tap Start and Complete on their phone, the customer signs off on their own phone, and an invoice is ready — all from one link, no paperwork."

1.2 Who To Pitch This To

GOOD FIT

Electricians, plumbers, AC/appliance repair, pest control, cleaning services, IT/networking installers, salon-at-home, equipment servicing — any business where staff travel to a customer's location to do the work.

NOT A FIT (YET)

Businesses needing live GPS tracking of staff throughout the day, or automatic nearest-technician dispatch/route planning — see Part 3 roadmap.

1.3 Pain Points This Solves

1 "We have no proof the technician actually went." Start/Complete are timestamped, and the technician's phone location is captured at both moments — a lightweight, honest proof-of-visit without

needing a tracking app.

- 2 **"Customers dispute whether the work was done or approved."** The customer signs off on their own phone, on their own booking link, with their typed name and a drawn signature — timestamped and stored against that exact job.
- 3 **"We lose track of parts/materials used on each job."** Technicians log materials used per job; if a material is a tracked product, stock is deducted automatically — no separate stock adjustment step.
- 4 **"Billing the customer after a job is a manual, delayed step."** Once a job is completed, "Convert to Invoice" builds the invoice from the service rate plus any priced materials in one click.

1.4 What's Included

FEATURE	WHAT THE CUSTOMER GETS
Technician Assignment	Every job can be assigned to a specific staff member at booking time or later.
Service Address	A free-text job-site address stored on the appointment, visible to the assigned technician.
Start / Complete Work	One-tap actions with timestamp + best-effort GPS snapshot, restricted to the assigned technician or a tenant admin.
Materials Used	Log parts/materials per job; product-linked entries auto-deduct from stock.
Before/After Photos	Technician uploads job photos, tagged Before or After — visible to the customer too.
Customer Sign-off	Digital signature capture (typed name + drawn signature) on the customer's own public booking link.
Convert to Invoice	One click turns a completed, priced job into an Invoice, linked back to the original appointment.
Public Self-Booking Link	Customers can still book appointments themselves via a shareable link (website/WhatsApp) — unchanged, still fully working.

1.5 Packaging & How To Grant Access

This entire workflow lives inside the existing **Appointments** module — it does not need separate packaging. Access works exactly like every other gated module in this CRM:

- 1 **Tenant's plan includes "Appointments / Booking"** — access turns on automatically.

2 **Superadmin grants it manually as an exception** — Superadmin → Tenants → open the tenant → Modules → Appointments → Enable. Useful for a demo or trial before the customer upgrades.

TWO MORE SWITCHES TO CHECK BEFORE DEMOING

Beyond module access, two tenant-level settings must also be on for the customer-facing side to work: (1) [Appointments → Booking Settings → "Enable public online booking"](#) , and (2) at least one [Service](#) marked **Bookable**. Both are off by default for a new tenant — walk the prospect through turning them on during the demo.

1.6 Objection Handling

OBJECTION	RESPONSE
"We need to see technicians moving on a live map."	Be upfront: today's version captures location only at Start and Complete, not continuously. Live map tracking is roadmap (see Part 3) — flag as "coming soon," don't oversell.
"How do we assign the nearest available technician automatically?"	Not built yet — assignment is a manual dropdown today, same as task assignment elsewhere in the CRM. Auto-dispatch/routing is roadmap.
"Do technicians need to install an app?"	No — everything runs in the phone's browser, including the Start/Complete geolocation capture. No app install, no app-store approval delay.



For End Users

Day-to-day usage guide — Tenant Admin and Technician

2.1 Booking a Job

- 1 Go to **Appointments** → **Book Appointment**.
- 2 Pick the Contact and the Service.
- 3 **New:** pick a Technician from the dropdown, and type the job-site address in Service Address.
- 4 Pick a date, duration, and time slot, then Book Appointment. You land straight on the new Job Detail page for that appointment.

2.2 The Job Detail Page

Every appointment now has a detail page at [Appointments → \(click any row\) → View](#). This is the single place to manage the whole job: contact/technician/address info, a shareable customer link, Start/Complete buttons, the materials list, photos, sign-off status, and the invoice link.

2.3 Starting a Job (Technician)

- 1 Open the job (from the Appointments list, or a link shared by the admin).
- 2 Tap **Start Work**. The browser may ask for location permission — allow it if asked, but it is not required to continue.
- 3 Status changes to **In Progress**, and the start time is recorded.

Only the technician assigned to this specific job, or a Tenant Admin, will see this button. Anyone else viewing the job sees a message explaining why they can't start it.

2.4 Completing a Job

- 1 On the same job page, under Complete Work, click + **Add Material** for each part/material used.
- 2 Pick a Product from the dropdown (its name and rate auto-fill) — or leave it as a custom item and type a name manually.
- 3 Enter the quantity used, and adjust the rate if needed.
- 4 Click **Complete Work**. Status changes to **Completed**; any product-linked material is deducted from stock immediately.

2.5 Uploading Photos

On the job page, under Before/After Photos, choose Before or After, select one or more image files, and Upload. Photos also show on the customer's public booking page.

2.6 Getting Customer Sign-off

Once a job is Completed, share the job's **Customer Link** (shown at the top of the job page) with the customer — via WhatsApp, SMS, or however you already send booking confirmations. On that page, the customer sees the job status and photos, types their name, draws their signature with a finger or mouse, and taps Approve & Sign. The job page then shows who signed and when.

2.7 Billing the Job

Once a job is Completed, click **Convert to Invoice** on the job page. An Invoice is created using the Service's rate plus any materials that had a rate entered, and you're taken straight to that Invoice.

2.8 The Public Self-Booking Link (unchanged)

Customers can still book their own appointments without staff involvement — nothing about this was changed by the Job Workflow update. Find the link at **Appointments** (top of the page, once enabled) — share it on your website or WhatsApp. The customer picks a service, picks an available time slot, fills in their name/phone/email, and the appointment appears in your Appointments list automatically.

IF THE LINK SHOWS "NO BOOKABLE SERVICES"

Go to each Service and turn on its "Bookable" flag, and make sure [Appointments → Booking Settings → Enable public online booking](#) is checked.



For Developers

Architecture, data model, decisions, and what's left

3.1 Design Decision

Rather than building a parallel "Job" or "Work Order" entity, this feature **extends the existing** `Appointment` **model**. Appointments already had a Contact, a Service, a time slot, a status lifecycle, and a public token-guarded customer link — the right shape for a field-service job. This avoided duplicating the booking/scheduling logic that already existed, and kept the feature inside the same `module:appointments` access gate with no extra gating work required.

3.2 Data Model Changes

appointments table — new columns

COLUMN	PURPOSE
<code>assigned_to</code>	FK to <code>users</code> , nullable — the assigned technician.
<code>service_address</code>	Free-text job-site address, nullable.
<code>work_started_at</code> / <code>_lat</code> / <code>_lng</code>	Timestamp + optional GPS snapshot captured on Start Work.
<code>work_completed_at</code> / <code>_lat</code> / <code>_lng</code>	Timestamp + optional GPS snapshot captured on Complete Work.
<code>materials_used</code>	JSON array — <code>{product_id?, name, quantity, rate?}</code> per row, same "items as JSON" convention used by Quotation/Invoice/PurchaseOrder.
<code>customer_signature</code> / <code>_signed_name</code> / <code>_signed_at</code>	Base64 PNG signature, typed name, and timestamp from the public sign-off.

<code>invoice_id</code>	FK to <code>invoices</code> , nullable — set once "Convert to Invoice" runs.
-------------------------	--

Also: `invoices.appointment_id` (nullable FK, mirrors the existing `quotation_id` column) and a new `appointment_attachments` table (mirrors `task_attachments` exactly, plus a `stage` enum `before / after`).

`status` is a plain string column (not a DB enum), so no migration was needed to add the new `in_progress` value — it's just a new entry in `Appointment::statuses()` .

New files

<code>app/Models/AppointmentAttachment.php</code>	Photo model — path, mime type, size, stage, <code>getUrlAttribute()</code> .
---	--

<code>app/Services/AppointmentJobService.php</code>	Lifecycle logic — <code>startWork()</code> , <code>completeWork()</code> , <code>signOff()</code> , <code>convertToInvoice()</code> . Mirrors the <code>WorkOrderService</code> style from the Manufacturing module.
---	--

3 migrations	Appointment job fields, invoice FK, attachments table.
--------------	--

3.3 Stock Deduction

`AppointmentJobService::completeWork()` deducts stock for each product-linked material via `Product::adjustStock(-qty)` — the same atomic, race-safe method (single SQL `UPDATE ... SET current_stock = GREATEST(0, current_stock + delta)`) built earlier for the Manufacturing module's Work Orders. No new stock-mutation code path was introduced.

3.4 Access Control

Two layers, both enforced server-side (not just hidden in the UI):

- 1 Module-level:** the entire `/appointments` route group is wrapped in `module:appointments` middleware — plan feature or superadmin override, same tri-state mechanism used by `manufacturing` , `time_tracking` , and `tickets` .
- 2 Job-level:** `AppointmentController::assertCanManageJob()` restricts Start/Complete to the assigned technician or a `tenant_admin` , via a 403 abort — verified with a direct unauthenticated-bypass-attempt test (a staff user with the `appointments.edit` permission but not assigned to the job gets HTTP 403 on a raw POST, not just a hidden button).

3.5 Geolocation — What It Is And Isn't

Location capture uses the browser's native `navigator.geolocation.getCurrentPosition()` — a one-time snapshot taken when Start Work or Complete Work is clicked, with a 4-second timeout. If permission is

denied, unavailable, or times out, the action proceeds anyway with null coordinates — geolocation never blocks the workflow.

NOT BUILT

This is not continuous/live tracking. There is no background location, no route history, and no "where is my technician right now" map. That would require either a native mobile app or a PWA with background location permission — a materially larger feature, intentionally out of scope here.

3.6 End-to-End Verification Performed

- Full `php artisan test` suite: 180/180 passing after the change (no regressions).
- Playwright walkthrough: book → assign technician + address → Start Work (with geolocation) → add a product-linked material → Complete Work → verified stock decremented by exactly the logged quantity → customer signed off on the public link → Convert to Invoice produced the correct line items and total.
- Negative-permission test: a staff user with `appointments.edit` permission but not assigned to the job — Start Work button not rendered, and a direct POST to `/appointments/{id}/start` returned HTTP 403 with the expected message.
- Public self-booking flow (pre-existing, unmodified except for a new sign-off section) re-verified end-to-end: anonymous browser → pick service → pick slot → fill details → submit → booking appears in the tenant's list with `source = public`.
- Fixed a pre-existing route-ordering bug found during this work: `GET /appointments/{id}` was registered before `GET /appointments/create` in the same route group, which would have made Laravel match `/appointments/create` against the show route (`id = "create"`) instead of the create route. Reordered so literal paths are matched first.

3.7 Roadmap / Not Yet Built

ITEM	PRIORITY	NOTES
Live GPS tracking of technicians	High interest	Needs a native app or background-location PWA — a separate project, not an extension of the current snapshot approach.
Auto-dispatch / nearest-technician assignment	Medium	Assignment is a manual dropdown today, consistent with Task assignment elsewhere in the CRM.

Route planning / multi-job scheduling optimization	Medium	Not started.
Recurring/scheduled maintenance jobs	Low	Not started — every job is currently booked individually.
Technician mobile push notifications on assignment	Low	Email/WhatsApp confirmation exists for the customer side (booking confirmation); no equivalent "you've been assigned a job" alert to the technician yet.

Field-Service Job Workflow — internal documentation. Extends the existing Appointments module; no new top-level module or permission set was introduced.